

Applied Machine Learning in Physics

Homework 1 (Sep. 12, 2026)

PHYS 2550
Due: Sep. 19, 2026

Instructions

Build a linear regression model from scratch, the way we did it in class: write the loss, derive the gradient, code the update. Use plain Python with NumPy and matplotlib. No scikit-learn, no PyTorch, no `np.polyfit` or similar – the point is that you write the fit yourself.

Hand in a short PDF report on Canvas with your derivations, plots, and the numbers you are asked for, plus a link to the notebook (Colab or GitHub) that produces them. The notebook has to run from the top without errors. 100 points, plus a 15-point bonus. Late submissions lose 5% per day.

Start early. The first part is quick, the rest is not.

House prices with linear regression

The data set is small (414 houses in Taipei, 6 numbers per house) and the model is the simplest one there is. That is on purpose: every step should be something you can check by hand.

(a) Get the data in (25 points)

Download the “Real estate valuation data set” from

<https://archive.ics.uci.edu/dataset/477/real+estate+valuation+data+set>. It comes as an XLSX file; convert it to CSV and say in one sentence how you did it. A copy is also here in case the site is down: https://drive.google.com/drive/folders/1PFy-5hFnq7UtAzsSxd6iatrVsG_FGIMQ?usp=sharing

Read the CSV in Python. Print the first few rows and the number of rows. For each column write one line: what it is, its units, its range. Look at the target (price per unit area) and at least two features with histograms. Anything odd? Missing values, a few houses far from everything else, a feature with a strange scale – write down what you see. You do not have to fix it yet, but you will need to remember it in (c).

(b) One feature (25 points)

Take house age as the only feature x and price as the target y . The model is

$$\hat{y} = w_0 + w_1 x.$$

Write the mean squared error loss, derive $\partial L / \partial w_0$ and $\partial L / \partial w_1$ by hand (show the two lines of algebra), and code gradient descent with those two expressions. Nothing else: a loop, two updates per iteration.

Report:

- the learning rate and number of epochs you used, and how you chose them;
- loss against epoch (log scale on y helps), showing that it converges;
- the final w_0 , w_1 and loss;
- a scatter plot of predicted against true price.

Is house age a good predictor? Say what the plot tells you, in two or three sentences.

(c) More features (25 points)

Add the distance to the nearest MRT station, and if you like one more feature:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 (+ \dots).$$

Write the model and the gradient in vector form, $\hat{y} = \mathbf{w}^\top \mathbf{x}$ with a constant 1 appended to $\mathbf{x}$, and rewrite your code so that it works for any number of features.

Distance to MRT is in metres and runs to several thousand; house age runs to about 40. Try gradient descent on the raw features first and see what happens. Then normalize the features (subtract the mean, divide by the standard deviation) and try again. Show both loss curves, and explain what went wrong the first time and why normalizing fixes it. Report the final weights and loss, and compare the loss to (b).

(d) What did we learn (25 points)

Compare the one-feature and the multi-feature model. Useful things to show: final loss of each; predicted vs. true price for each, on the same axes; a 2D histogram or scatter of price against each feature. Pick the ones that make your point.

Then answer, briefly:

- which features carry information about the price, and which do not;
- what limited you: outliers, the scale of the features, the learning rate, the number of epochs;
- what you would change in a second, more careful version.

Half a page is enough. A plot that says something beats a paragraph that does not.

Bonus (15 points)

For a linear model the best weights can be written down in closed form (the normal equation, $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$). Compute them for the model in (c) and compare with what gradient descent found. How close is close enough, and how many epochs did gradient descent need to get there? If your gradient descent does not reach the closed-form answer, find out why.